تحليل النظام

Date No.

" سيتم تقسيم النظام الى عمليات صغيره "

(A)

① سحب البيانات من موقع quotes " إقتباسات " بإستخدام BeautifulSoup

② تخزين البيانات المسحوبه مؤقتاً في list

③ عملية تنظيف البيانات :
Ⓐ تحويل البيانات الى DF Ⓑ تنظيف البيانات بإستخدام pandas

④ عملية تخزين البيانات النظيفه :
Ⓐ تخزين البيانات في json لسهولة التعامل معها في Ubuntu

⑤ عملية عرض البيانات النظيفة API :
Ⓐ EndPoints لعرض البيانات النظيفه من ملف الـ json

(B)

① تجهيز هيكل مجلد التنفيذ :
Ⓐ مجلد usr/local/bin : مسار وضع ملفات التنفيذ " المسار المعياري في لينكس "

② تجهيز ~~ملف~~ " مجلد " الخدمه :
Ⓐ مجلد etc/systemd/system : مسار وضع ملف الخدمه للتشغيل التلقائي عند التشغيل

③ ~~تجهيز~~ تجهيز مجلد التثبيت DEBIAN :
Ⓐ ملف control : اسم الخدمه ، وصف ، إصدار ، وال Depends والخ
Ⓑ ملف postinst : سكربت تنفيذ العمليات فور التثبيت مثل 8000 allow ufw والخ

④ ~~ملف~~ <= يتبع

تحليل النظام (2)

Date No.

④ ملف التثبيت النهائي

Ⓐ ملف ken_install.sh :

① تحديث قائمة الحزم apt update

② تثبيت ملف الـ deb وتحميل الـ Depends التي في الـ control

"سبب اختيار هذا التصميم "dataflow" "

① التتبع بدقة لدورة حياة البيانات :

Ⓐ Extraction : بما ان الموقع ثابت استخدمت Requests و BeautifulSoup لاستخراج البيانات

Ⓑ Transformation : اخترت Pandas لانها تعمل في الذاكرة فهي سريعة جداً

Ⓒ Loading : استخدمت sqlite3 للتخزين لانها سريعة للبيانات الصغيرة وسهلة التعامل في ubuntu

② إدارة دورة حياة التطبيق :

Ⓐ بدلاً من تشغيل التطبيق كسيرت عادي يعتمد التصميم على تحويله الى systemDaemon

① استخدمت enable --now و systemctl daemon-reload يضمن استمرار العمل بعد اعادة التشغيل

③ أتمتة التثبيت والتشغيل :

Ⓐ استخدمت ملف خارجي ken_install.sh لتحديث الحزم وتثبيت ملف الـ deb مما يسهل بشكل كبير

④ اخترت هذا التصميم لانه قائم على Decoupling وأتمتة الخدمات عبر ليناكس لضمان بناء نظام Production-Ready يتميز بالمرونة والامن .

★ ماذا ينقص النظام ؟ :

ينقصه التحققات Error handling وسيتم تعزيزه في مراحل التطوير القادمة لضمان موثوقية النظام